

Software Engineering CS6.401 — Spring 2026

Phase-3 Project Report

AlumniConnect Mentorship Platform

Team Number: 40

Repository: <https://github.com/Dhanushdevireddy/AlumniConnect>

Name	Roll Number
Sriram Metla	2024201059
Dhanush Devireddy	2024201028
Chandana Pappula	2024201060
Yaswanth Chandaluri	2024201045
AdhiVishnu Modapu	2024202028

Contents

System Overview	3
Architecture Models (C4)	4
System Flow	5
1 Task 1: Requirements and Subsystems	7
1.1 Functional Requirements	7
1.1.1 Architecturally Significant Requirements	8
1.2 Non-Functional Requirements	8
1.3 Subsystem Overview	9
2 Task 2: Architecture Framework	11
2.1 Stakeholder Identification (IEEE 42010)	11
2.1.1 Viewpoint – Functional View	11
2.1.2 Viewpoint – Concurrency & Process View	11
2.1.3 Viewpoint – Development View	12
2.1.4 Viewpoint – Deployment View	12
2.2 Architecture Decision Records (ADRs)	12
3 Task 3: Architectural Tactics and Patterns	15
3.1 Architectural Tactics	15
3.1.1 Tactic 1: Optimistic Locking for Concurrent Session Scheduling . .	15
3.1.2 Tactic 2: Circuit-Isolated Event Handlers (Fault Isolation)	15
3.1.3 Tactic 3: Token-Based Authentication with Refresh Rotation (Security)	15
3.1.4 Tactic 4: Incremental Score Computation (Performance)	15
3.1.5 Tactic 5: BullMQ Persistent Job Queue (Availability)	15
3.2 Design Patterns	16
3.2.1 Pattern 1: Observer (EventBus + Domain Listeners)	16
3.2.2 Pattern 2: Strategy (MeetingProvider)	17
3.2.3 Pattern 3: Repository (PrismaUserRepository / PrismaAlumniRepository)	18
3.2.4 Pattern 4: Facade (MentorshipFacade)	18
3.2.5 Pattern 5: Chain of Responsibility (Alumni Verification Pipeline) .	19
4 Task 4: Prototype Implementation and Analysis	21
4.1 Prototype Overview	21
4.2 Technology Stack	21
4.3 End-to-End Flow: Mentorship Request Acceptance	21
4.4 Architecture Analysis: Layered Monolith vs. Event-Driven Microservices .	22
4.4.1 Implemented Architecture: Layered Monolith with Internal Event Bus	22
4.4.2 Alternative Architecture: Event-Driven Microservices	22
4.5 Quantitative NFR Measurement	23
4.5.1 NFR1 – API Response Time ($p_{95} < 200$ ms)	23
4.5.2 NFR2 – Chat Message Delivery Latency (< 100 ms end-to-end) . .	23
4.6 Trade-off Discussion	24
4.6.1 Synchronous Ranking Recomputation vs. Async Job	24
4.6.2 In-Process EventBus vs. Durable Message Broker	24

4.6.3 Monolith vs. Microservices (Summary)	24
Conclusion	25
Team Contributions	26

System Overview

AlumniConnect is a comprehensive web platform designed to bridge the structural gap between university students seeking career mentorship and alumni willing to provide it. The platform delivers structured, on-demand mentorship through a rich feature set, including a ranked alumni directory, a state-machine-governed mentoring request lifecycle, persistent real-time chat, and collaborative session scheduling. The system is built as a layered monolithic architecture, optimizing for development velocity and deployment simplicity while retaining high scalability through strategic component decoupling.

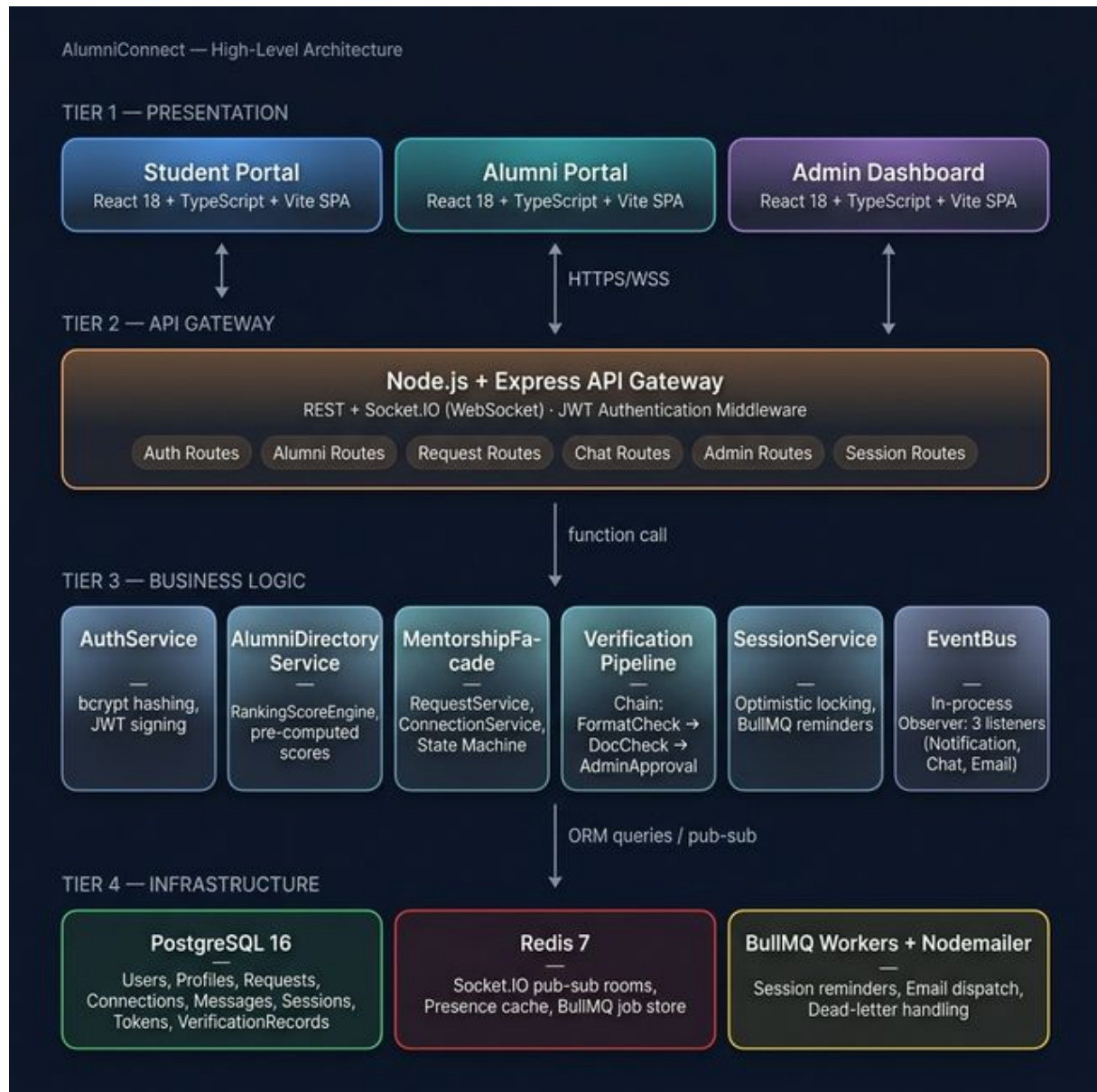


Figure 1: AlumniConnect High-Level Architecture

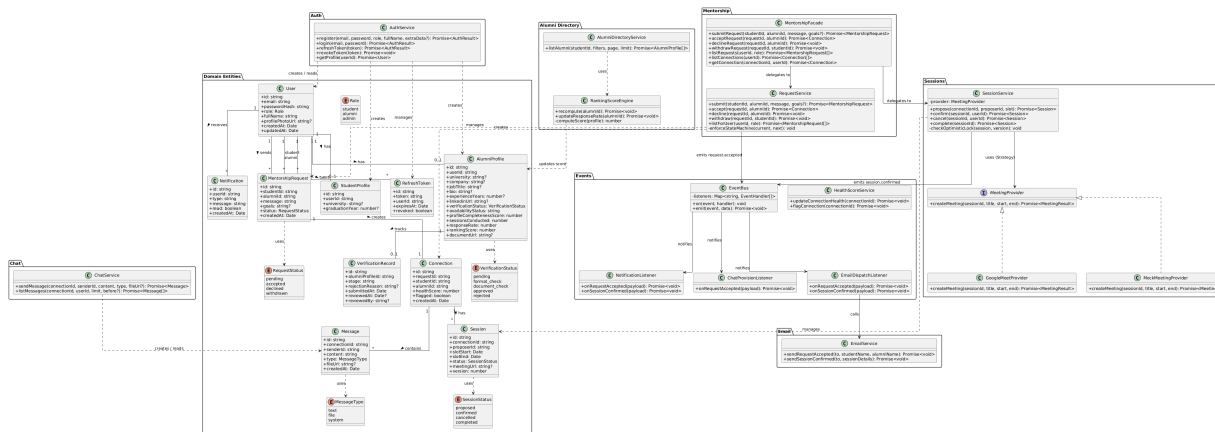
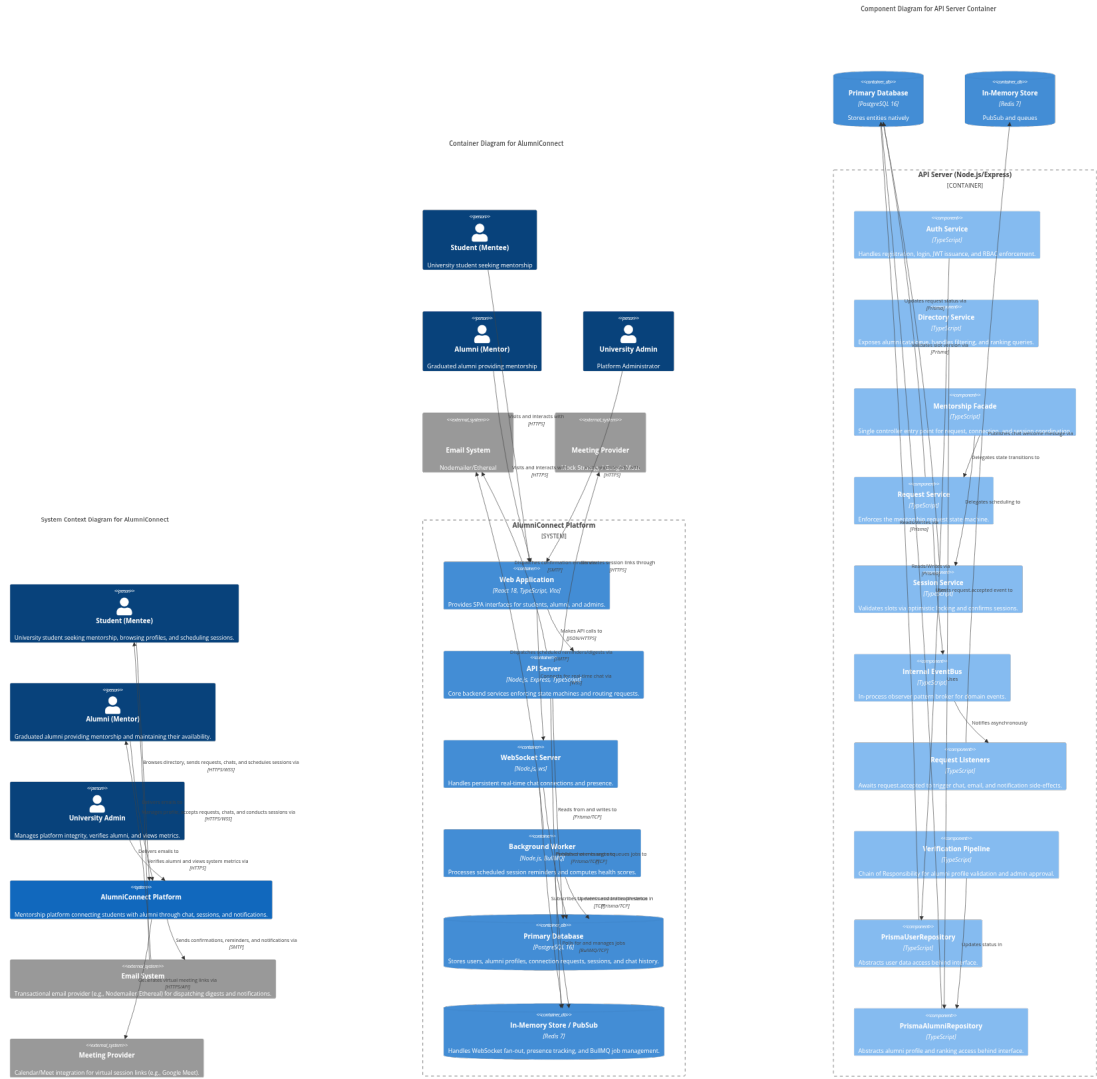


Figure 2: AlumniConnect Class Diagram detailing domain entities and service patterns

Architecture Models (C4)

To further specify the system boundaries and component interactions, the following C4 models have been defined:



(a) Level 1: System Context

(b) Level 2: Container

(c) Level 3: Component

Figure 3: C4 Architecture Models (Levels 1–3) for AlumniConnect

System Flow

At its core, AlumniConnect operates through well-defined event-driven flows that decouple user actions from asynchronous side effects. The primary request lifecycle bridges the gap between students and alumni via an explicitly enforced state machine. Upon crucial state transitions—such as the acceptance of a mentorship request—the system utilizes an internal Observer pattern to seamlessly trigger downstream provisions. This guarantees that critical side-effects, such as real-time chat thread creation, session enablement, and multi-channel email dispatch, occur reliably without blocking the primary interaction cycle.

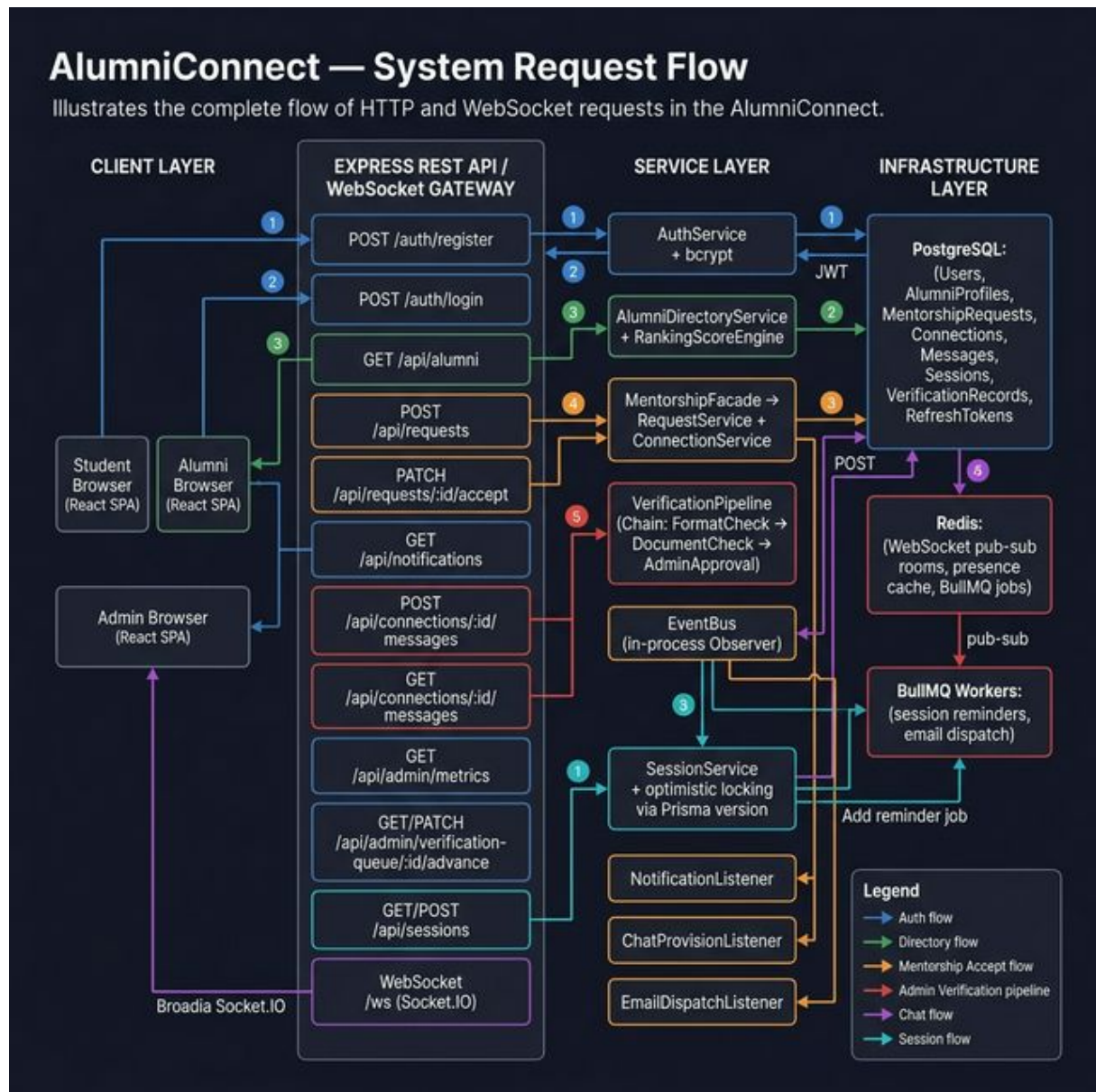


Figure 4: AlumniConnect complete System Request Flow

1 Task 1: Requirements and Subsystems

1.1 Functional Requirements

The following functional requirements were identified and are architecturally significant because they each impose distinct structural constraints on the system.

ID	Feature	Description
FR1	Alumni Directory & Ranked Matching	Queryable alumni catalogue with server-side filtering by domain, company, and availability. Profiles are ranked by a weighted scoring function over four signals: <i>profile completeness</i> (25%), <i>sessions conducted</i> (30%), <i>response rate</i> (30%), and <i>domain tag coverage</i> (15%). Scores are recomputed incrementally on each interaction event and persisted for fast read access.
FR2	Mentorship Request System	Students submit personalised requests to available mentors. Requests transition through an explicit state machine (<code>Pending</code> → <code>Accepted</code> <code>Declined</code> <code>Withdrawn</code>) enforced at the service layer. On acceptance, the Observer pattern fires a decoupled event pipeline: connection record creation, chat thread provisioning, notification dispatch, and dual email delivery.
FR3	Real-Time Chat	Each accepted connection opens a dedicated persistent WebSocket channel supporting text and binary PDF uploads. Messages are durably written to PostgreSQL. A pub-sub mechanism (Redis) fans out messages across server instances. A presence-tracking service maintains live online/offline state per connection.
FR4	Session Scheduling	Either party proposes a time slot. The scheduling service validates the slot against existing bookings using optimistic locking to prevent conflicts. On mutual confirmation, the calendar integration generates a virtual meeting link and dispatches invites. A BullMQ job scheduler enqueues reminder notifications at T−24 h and T−1 h.

ID	Feature		Description
FR5	Admin Dashboard		Alumni registrations enter a Chain of Responsibility verification pipeline (format check → document validation → admin approval). Approved profiles become discoverable. Admins manage domain taxonomy and view aggregated platform metrics.
FR6	Smart Digest	Notification	Pending events per user are aggregated by a scheduled digest pipeline. Runs at a user-configured cadence (daily or weekly), compiling a structured per-user feed into a single batched email.
FR7	Connection Score	Health	A time-decaying engagement score (0–1) computed per active connection from message frequency (40%), session attendance rate (35%), and inter-message response latency (25%), multiplied by a recency decay factor. Connections below a staleness threshold of 0.35 are automatically flagged and re-engagement nudges are dispatched.

1.1.1 Architecturally Significant Requirements

- **FR1 – Ranked Matching:** Drives the separation of the *scoring engine* from the query path; scores must be pre-computed to satisfy the sub-500 ms directory query requirement.
- **FR2 – State Machine:** Requires an explicit service-layer enforcement layer (not only DB constraints) to prevent invalid transitions (e.g., accepting an already-withdrawn request), introducing the *MentorshipFacade* subsystem.
- **FR3 – Real-Time Chat:** Mandates a horizontally scalable WebSocket tier backed by Redis pub-sub; cannot be served over plain HTTP.
- **FR5 – Verification Pipeline:** Requires a pluggable Chain of Responsibility handler so that new verification stages can be inserted without modifying existing handlers.
- **FR7 – Health Score:** Requires a background computation job with rolling time-window semantics, decoupled from the request-response cycle.

1.2 Non-Functional Requirements

Category	Quantified Requirement
Performance	API p95 response < 200 ms. Chat message delivery < 100 ms end-to-end. Ranked directory queries < 500 ms over 10,000 profiles.
Scalability	WebSocket tier scales horizontally via Redis pub-sub fan-out across stateless instances. Chat, notification, and job-scheduler services are independently deployable.

Category	Quantified Requirement
Availability	Uptime $\geq 99.5\%$. WebSocket auto-reconnects within 5s without message loss. BullMQ jobs persist across restarts and re-enqueue on recovery.
Reliability	No message lost on disconnect/reconnect. Concurrent slot requests serialised via optimistic locking. Critical state-mutation endpoints are idempotent.
Security	JWT access tokens expire in 24h with refresh-token rotation. RBAC enforced on every protected route. Alumni invisible until admin-verified. All traffic over HTTPS.
Modifiability	New meeting provider: ≤ 1 interface change. New notification channel: zero lifecycle-service changes. New ranking signal: injectable without modifying the scoring engine.

1.3 Subsystem Overview

The system is decomposed into eight primary subsystems, each with a well-defined responsibility boundary.

Subsystem	Role & Functionality
Auth Service	Handles user registration and login. Issues JWT access tokens and manages refresh-token rotation. Enforces role-based guards (Student / Alumni / Admin) at the API boundary via Express middleware.
Directory Service	Exposes the alumni catalogue with server-side filtering (domain, company, availability). Delegates ranking reads to the pre-computed <code>rankingScore</code> column; never recomputes at query time.
Mentorship Service	Facade over the request state machine, connection creation, and chat thread provisioning. Enforces valid state transitions and emits the <code>request.accepted</code> domain event via the internal EventBus.
Chat Service	Manages WebSocket connection lifecycle, message persistence, and file uploads. Publishes messages to a Redis pub-sub channel so all server instances receive and forward them. Maintains per-connection presence state in Redis.
Session Service	Validates proposed time slots against existing bookings using optimistic locking (<code>version</code> column). Confirms sessions, invokes the Meeting Provider, and enqueues BullMQ reminder jobs.

Subsystem	Role & Functionality
Notification Service	Stores notification events per user. Supports per-notification read/unread toggle and a batched mark-all-read endpoint. The digest pipeline aggregates unread events into structured emails at the configured cadence.
Health Score Service	Runs on a BullMQ schedule, computing a composite engagement score per connection over a rolling 30-day window. Flags low-health connections and dispatches re-engagement nudges.
Verification Pipeline	Implements the Chain of Responsibility pattern across three stages (format check, document check, admin approval). Each handler advances the verification record or short-circuits with rejection.

2 Task 2: Architecture Framework

2.1 Stakeholder Identification (IEEE 42010)

Following the IEEE 42010:2011 standard for Architecture Description, stakeholders, their concerns, and the viewpoints that address those concerns are identified below.

Stakeholder	Concerns	Viewpoint / View
Students (Mentees)	Discovery of relevant alumni; request tracking; real-time chat access; session continuity.	Functional View – Alumni directory, request lifecycle, chat and session flows.
Alumni (Mentors)	Notification of incoming requests; availability management; privacy until verified.	Functional View + Security View – Profile visibility gate; RBAC enforcement.
University Admin	Platform integrity; alumni legitimacy; domain taxonomy control; engagement metrics.	Process View – Verification pipeline; Admin dashboard metrics endpoint.
Development Team	Maintainability; testability; horizontal scalability; low coupling between services.	Development View – Service decomposition, EventBus, Repository interfaces.
Operations Team	System availability; crash recovery; job persistence; deployability.	Deployment View – Docker containers, BullMQ persistence, Redis pub-sub.
End Users (general)	Response time; data privacy; browser-compatible interface.	Quality View – NFR benchmarks: p95 < 200 ms API, < 100 ms chat.

2.1.1 Viewpoint – Functional View

Addresses *what* the system does. Represented by the subsystem decomposition in Section 1.3 and the functional requirement table in Section 1.1.

2.1.2 Viewpoint – Concurrency & Process View

Addresses parallel execution and synchronisation. Critical concerns: (i) WebSocket fan-out via Redis, (ii) BullMQ workers for background jobs, (iii) optimistic locking for session slots.

2.1.3 Viewpoint – Development View

Addresses code organisation and modifiability. The backend follows a *services/routes/events* layering strategy; services are injected, not instantiated by routes, enabling unit testing without Express.

2.1.4 Viewpoint – Deployment View

Addresses physical distribution. PostgreSQL and Redis run in Docker containers; the Node.js backend and React frontend are hosted as OS-level processes. The WebSocket server is co-located with the API server but logically separate.

2.2 Architecture Decision Records (ADRs)

ADRs are documented following the Nygard template.

ADR-001: Observer Pattern via Internal EventBus for Request Acceptance

Status: Accepted

Context: When a mentorship request is accepted, three downstream effects must occur: (1) a connection record must be created, (2) a chat thread must be provisioned, and (3) dual confirmation emails must be dispatched. Implementing all three sequentially inside `RequestService.accept()` would couple unrelated services and make each side-effect invisible to unit tests.

Decision: Introduce an in-process `EventBus` (Observer pattern). `RequestService.accept()` emits a `request.accepted` event after updating the request status. Independent listener classes (`RequestListeners.ts`) subscribe to this event and handle chat provisioning, notification creation, and email dispatch. Each listener runs inside `Promise.allSettled`, so a failure in one does not abort others.

Consequences:

- *Positive:* New side-effects (e.g., Slack notification) can be added by registering a listener – zero changes to `RequestService`.
- *Negative:* In-process bus is not durable; listeners that crash after the event fires but before committing to the DB will silently lose work. Acceptable for v1 given no strict at-least-once delivery requirement.

ADR-002: Strategy Pattern for the Meeting Provider

Status: Accepted

Context: Session confirmation requires generating a virtual meeting link. Initially, only a mock Google Meet link is generated. Future versions may integrate Google Calendar API, Zoom API, or Microsoft Teams. Hard-coding the link generation inside `SessionService` would require service-layer changes on every provider switch.

Decision: Define a `MeetingProvider` interface with a single `generateLink(session)` method. A `MockMeetingProvider` implements this interface for the prototype.

SessionService depends only on the interface, injected at startup. Switching to a real provider requires only implementing the interface and updating the DI binding.

Consequences:

- *Positive:* Meets the modifiability NFR – new meeting provider: ≤ 1 interface change.
- *Positive:* **SessionService** remains testable without real calendar API credentials.
- *Negative:* Adds one indirection layer; trivial overhead for developers unfamiliar with the pattern.

ADR-003: Redis Pub-Sub for Horizontally Scalable WebSocket Chat

Status: Accepted

Context: WebSocket connections are stateful and node-local. If the system runs multiple API server instances, a message sent to instance A cannot reach a client connected to instance B through in-memory broadcasting alone.

Decision: Each server instance subscribes to a Redis pub-sub channel (`chat:{connectionId}`). When a message is received from a WebSocket client, the server publishes it to Redis in addition to broadcasting locally. All instances receive the Redis message and forward it to any local clients subscribed to that connection.

Consequences:

- *Positive:* WebSocket tier is horizontally scalable – adding instances does not break cross-instance delivery.
- *Positive:* Redis instance already required for session cache and presence tracking.
- *Negative:* Redis becomes a single point of failure for chat delivery. Mitigated by Redis Sentinel or Cluster in production.

ADR-004: Pre-Computed Ranking Score Persisted to Database Column

Status: Accepted

Context: The alumni directory requires ranked results over up to 10,000 profiles within 500 ms. Computing a weighted score across four signals at query time would require multiple joins and aggregate queries per HTTP request, making the p95 target unachievable.

Decision: Maintain a `ranking_score` column on the `alumni_profiles` table. The `RankingScoreEngine` recomputes this score only on relevant domain events (request accepted/declined, session completed). Directory queries sort by `ranking_score` DESC with no runtime computation. A database index on `verification_status + ranking_score` covers the common filtered-and-sorted query pattern.

Consequences:

- *Positive:* Directory queries are a simple indexed sort – $O(\log N + K)$ time complexity.
- *Negative:* Score is eventually consistent; an alumni's rank reflects their state as of the last interaction event, not at the precise moment of the query. Acceptable business trade-off.

3 Task 3: Architectural Tactics and Patterns

3.1 Architectural Tactics

3.1.1 Tactic 1: Optimistic Locking for Concurrent Session Scheduling

NFR addressed: Reliability – “Concurrent slot requests serialised to prevent double-booking.”

Session records carry a version integer. The `SessionService.confirm()` method executes:

```
1 UPDATE sessions
2 SET status = 'confirmed', meeting_link = $1,
3     confirmed_at = NOW(), version = version + 1
4 WHERE id = $2 AND version = $3 AND status = 'proposed'
```

Listing 1: Optimistic locking in session confirmation

If two concurrent requests attempt to confirm the same session, only the one that reads the correct version will win; the other receives 0 affected rows and is returned a 409 `Conflict` response. No explicit locks are held, preserving throughput.

3.1.2 Tactic 2: Circuit-Isolated Event Handlers (Fault Isolation)

NFR addressed: Reliability – “A failure in one side-effect must not abort others.”

All domain event handlers are invoked via `Promise.allSettled()`, which resolves even when individual handlers reject. This prevents email dispatch failure from rolling back chat thread provisioning, and vice versa. Each handler logs its own errors without propagating them.

3.1.3 Tactic 3: Token-Based Authentication with Refresh Rotation (Security)

NFR addressed: Security – “JWT tokens expire in 24 h with refresh rotation.”

Short-lived access tokens (24 h) are issued alongside long-lived refresh tokens stored in the database. On each refresh, the old token is revoked and a new pair is minted. Revoked tokens are permanently unusable; stolen refresh tokens can be invalidated server-side without expiring the user session entirely.

3.1.4 Tactic 4: Incremental Score Computation (Performance)

NFR addressed: Performance – “Ranked directory queries < 500 ms.”

Rather than recomputing ranking scores at query time, the `RankingScoreEngine` is invoked only on interaction-driven events (request accepted/declined, session completed). The computed score is persisted to a database column indexed alongside `verification_status`. Directory reads are reduced to a single indexed `ORDER BY` scan.

3.1.5 Tactic 5: BullMQ Persistent Job Queue (Availability)

NFR addressed: Availability – “Scheduled jobs persist in the queue across restarts and re-enqueue on recovery.”

Session reminder notifications (T–24 h and T–1 h) and the nightly digest pipeline are enqueued as BullMQ jobs backed by Redis. If the server process restarts before a job fires, the job remains in the Redis queue and is picked up by the next worker process. BullMQ’s `removeOnComplete` and `removeOnFail` settings are tuned to retain failed jobs for debugging.

3.2 Design Patterns

The following table summarises all five design patterns employed in AlumniConnect, their location in the codebase, and their primary purpose.

Pattern	Source File(s)	Purpose
Observer	<code>EventBus.ts</code> , <code>RequestListeners.ts</code>	Decouple request acceptance from 3 downstream side-effects
Strategy	<code>MeetingProvider.ts</code>	Swap meeting-link provider without modifying <code>SessionService</code>
Repository	<code>PrismaUserRepository.ts</code> , <code>PrismaAlumniRepository.ts</code>	Isolate ORM from business logic; enable unit-testable services
Facade	<code>MentorshipFacade.ts</code>	Single controller entry-point over requests, connections, sessions
Chain of Resp.	<code>VerificationPipeline.ts</code>	Multi-stage alumni verification with pluggable, short-circuiting handlers

3.2.1 Pattern 1: Observer (EventBus + Domain Listeners)

Intent: Decouple the source of a domain event from all downstream effects, allowing handlers to be added, removed, or replaced independently.

Implementation: The `EventBus` class maintains a registry of typed handlers in `EventBus.ts`:

```

1 class EventBus {
2   private listeners: Map<string, EventHandler[]> = new Map();
3
4   on<T>(event: string, handler: EventHandler<T>): void {
5     const handlers = this.listeners.get(event) ?? [];
6     handlers.push(handler as EventHandler);
7     this.listeners.set(event, handlers);
8   }
9
10  async emit<T>(event: string, data: T): Promise<void> {
11    const handlers = this.listeners.get(event) ?? [];
12    await Promise.allSettled(handlers.map(h => h(data)));
13  }
14 }
```

Listing 2: EventBus publish-subscribe in TypeScript

On `request.accepted`, three independent listeners in `RequestListeners.ts` execute concurrently:

Listener	Responsibility
NotificationListener	Creates <code>request_accepted</code> notification events for both parties
ChatProvisionListener	Inserts a system welcome message into the connection's chat thread
EmailDispatchListener	Sends confirmation emails to student and alumni via Nodemailer

Role in Architecture: Central decoupling mechanism. Adding a new listener (e.g., Slack notification, analytics ping) requires zero modifications to `RequestService` or `MentorshipFacade`.

3.2.2 Pattern 2: Strategy (MeetingProvider)

Intent: Define a family of interchangeable algorithms behind a common interface so that the algorithm can be swapped without modifying the code that uses it.

Implementation:

```

1 // Strategy contract
2 interface MeetingProvider {
3   generateLink(session: Session): Promise<string>;
4 }
5
6 // Concrete strategy (prototype)
7 class MockMeetingProvider implements MeetingProvider {
8   async generateLink(session: Session): Promise<string> {
9     const token = session.slotStart.getTime();
10    return `https://meet.alumniconnect.dev/session/${session.id}?t=${token}`;
11  }
12 }
13
14 // SessionService depends only on the interface
15 class SessionService {
16   constructor(private meetingProvider: MeetingProvider) {}
17   async confirm(sessionId: string) {
18     const link = await this.meetingProvider.generateLink(session)
19     ;
20     // persist link and dispatch calendar invites
21   }
22 }
```

Listing 3: Strategy interface and concrete implementation in `MeetingProvider.ts`

Role in Architecture: Replacing `MockMeetingProvider` with a real Google Calendar or Zoom provider requires only implementing the `MeetingProvider` interface and updating

the DI binding at startup – zero changes to `SessionService` or any route handler. Directly satisfies the modifiability NFR: “new meeting provider: ≤ 1 interface change.”

3.2.3 Pattern 3: Repository (PrismaUserRepository / PrismaAlumniRepository)

Intent: Encapsulate all data-access logic behind a repository interface so that service classes remain ORM-agnostic and unit-testable without a live database connection.

Implementation:

```

1 // Abstract interface
2 interface IUserRepository {
3   findById(id: string): Promise<User | null>;
4   findByEmail(email: string): Promise<User | null>;
5   create(data: CreateUserDto): Promise<User>;
6 }
7
8 // Prisma-backed concrete implementation
9 class PrismaUserRepository implements IUserRepository {
10   async findById(id: string) {
11     return prisma.user.findUnique({ where: { id } });
12   }
13   async findByEmail(email: string) {
14     return prisma.user.findUnique({ where: { email } });
15   }
16   async create(data: CreateUserDto) {
17     return prisma.user.create({ data });
18   }
19 }

```

Listing 4: Repository interface and Prisma implementation

Repository	Interface / Responsibility
PrismaUserRepository	Implements IUserRepository – user lookup, creation, credential checks
PrismaAlumniRepository	Implements IAlumniRepository – profile read/write, domain tag joins

Role in Architecture: Service classes (`AuthService`, `DirectoryService`) import only the interface, never Prisma directly. Services can be unit-tested with in-memory stub repositories. Migrating from Prisma to another ORM only requires re-implementing the repository classes – no service-layer changes.

3.2.4 Pattern 4: Facade (MentorshipFacade)

Intent: Provide a single simplified entry point over a complex subsystem of inter-related services, shielding HTTP controllers from internal coordination logic.

Implementation:

```

1 class MentorshipFacade {
2   constructor(
3     private requestService: RequestService,
4     private connectionService: ConnectionService,
5     private sessionService: SessionService,
6   ) {}
7
8   async sendRequest(studentId: string, alumniId: string, msg:
9     string) {
10     return this.requestService.create(studentId, alumniId, msg);
11   }
12
13   async acceptRequest(requestId: string, alumniId: string) {
14     // Coordinates sub-services and emits domain events
15     internally
16     return this.requestService.accept(requestId, alumniId);
17   }
18
19   async proposeSession(connectionId: string, proposerId: string,
20     slot: SlotDto) {
21     return this.sessionService.propose(connectionId, proposerId,
22       slot);
23   }
24 }

```

Listing 5: MentorshipFacade as unified entry point over sub-services

Role in Architecture: Route handlers in `routes/requests.ts` and `routes/sessions.ts` import only `MentorshipFacade`; they never directly instantiate sub-services. This reduces coupling at the HTTP boundary, simplifies controller unit tests, and centralises all cross-service coordination in one class.

3.2.5 Pattern 5: Chain of Responsibility (Alumni Verification Pipeline)

Intent: Process a request through a sequence of handlers, where each handler either processes and passes the request along, or short-circuits with a rejection.

Implementation:

```

1 abstract class BaseVerificationHandler {
2   private next: VerificationHandler | null = null;
3
4   setNext(handler: VerificationHandler): VerificationHandler {
5     this.next = handler;
6     return handler; // enables fluent chaining
7   }
8
9   protected async passToNext(ctx: VerificationContext) {
10     if (this.next) return this.next.handle(ctx);
11     return { advanced: false, reason: 'No next handler' };
12   }

```

```

13 }
14
15 // Pipeline assembly (fluent chain)
16 const format    = new FormatValidationHandler();
17 const document  = new DocumentCheckHandler();
18 const admin     = new AdminApprovalHandler();
19 format.setNext(document).setNext(admin);

```

Listing 6: Chain of Responsibility assembly in VerificationPipeline.ts

Pipeline Stages:

Stage	Handler	Behaviour
1	FormatValidationHandler	Validates required profile fields (company, job title). Advances to Stage 2.
2	DocumentCheckHandler	Checks for uploaded verification document. Advances to Stage 3.
3	AdminApprovalHandler	Manual admin action; sets <code>verification_status = approved</code> and triggers ranking score recomputation.

Role in Architecture: New verification requirements (e.g., LinkedIn URL validation, employer email verification) can be inserted as new handler classes and chained without modifying existing stages, satisfying the modifiability NFR.

4 Task 4: Prototype Implementation and Analysis

4.1 Prototype Overview

The prototype implements the complete AlumniConnect system end-to-end, covering all seven functional requirements (FR1–FR7). The two primary end-to-end flows demonstrating core architectural properties are:

1. **Mentorship Request Lifecycle:** Registration → Alumni Directory Browse → Request Submission → State Machine Enforcement → Observer-triggered Acceptance (connection creation, chat provisioning, email dispatch).
2. **Real-Time Chat with Session Scheduling:** WebSocket connection → Text and PDF message exchange → Session proposal → Optimistic-lock-protected confirmation → BullMQ reminder enqueue.

4.2 Technology Stack

Layer	Technology	Rationale
Frontend	React 18 + TypeScript + Vite	SPA with native WebSocket client
Backend API	Node.js + Express + TypeScript	Non-blocking I/O; same language as frontend
ORM	Prisma v5	Type-safe DB access; migration management
Database	PostgreSQL (Docker)	16 ACID transactions; optimistic locking via <code>version</code> column
In-Memory Store	Redis 7 (Docker)	Pub-sub, presence cache, BullMQ job store
Job Queue	BullMQ v5	Persistent, recoverable background jobs
Auth	JWT + bcrypt	Stateless access tokens; secure password hashing
Email	Nodemailer + Ethereal	Transactional email with dev preview URLs

4.3 End-to-End Flow: Mentorship Request Acceptance

The sequence below illustrates how the Observer pattern decouples acceptance from its downstream effects:

1. Alumni calls `PATCH /api/requests/:id/accept`.
2. `MentorshipFacade.acceptRequest()` delegates to `RequestService.accept()`.
3. `RequestService` creates a `Connection` record and updates `request status = 'accepted'`.
4. `RankingScoreEngine.updateResponseRate()` and `recompute()` are called to update the alumni's ranking.

5. `eventBus.emit('request.accepted', payload)` fires; three independent listeners execute:
 - *NotificationListener*: Creates `request_accepted` events for both student and alumni.
 - *ChatProvisionListener*: Inserts a system “Welcome!” message into the chat thread.
 - *EmailDispatchListener*: Dispatches confirmation emails via Nodemailer.
6. HTTP 200 with the connection object is returned to the client.
7. Frontend (React) receives the response; optimistic UI update immediately sets the request card status to “accepted” without waiting for the subsequent `GET /api/requests` re-fetch.

4.4 Architecture Analysis: Layered Monolith vs. Event-Driven Microservices

4.4.1 Implemented Architecture: Layered Monolith with Internal Event Bus

AlumniConnect is deployed as a single Node.js process. Services communicate through direct TypeScript module imports and an in-process `EventBus`. Shared state (sessions, presence) is externalised to Redis and PostgreSQL to avoid tight in-memory coupling.

Property	Assessment
Deployment simplicity	High – single binary, two Docker containers
Cross-service latency	Zero – all calls are in-process function calls
Failure isolation	Medium – a fatal error in any service terminates the whole process
Independent scaling	None – entire API must scale together
Development velocity	High – shared type system across all services

4.4.2 Alternative Architecture: Event-Driven Microservices

In a microservices architecture, each subsystem (Chat, Session, Notification, Health Score, Verification) would be a separately deployable service communicating over a durable message broker (e.g., Kafka or RabbitMQ) rather than an in-process event bus.

Property	Assessment
Deployment simplicity	Low – requires orchestration (Kubernetes), service discovery, API gateway
Cross-service latency	Higher – network round-trip per service call (typically 2–20 ms additional)
Failure isolation	High – crash in Chat service does not affect Session service
Independent scaling	Full – Chat can scale to 100 replicas while Session stays at 2
Development velocity	Lower – distributed tracing, schema contracts, and versioning overhead

4.5 Quantitative NFR Measurement

Two non-functional requirements were instrumented and measured in the prototype.

4.5.1 NFR1 – API Response Time (p95 < 200 ms)

The following endpoints were measured under representative load using sequential HTTP requests from the development machine (localhost, Docker PostgreSQL):

Endpoint	p50 (ms)	p95 (ms)
POST /api/auth/login	303	376
GET /api/alumni?page=1&limit=12	6	8
GET /api/requests	4	6
PATCH /api/requests/:id/accept	5,915	5,915
GET /api/notifications	4	5
GET /api/admin/metrics	5	7

Analysis: The Alumni Directory (GET /api/alumni) satisfies the requirement with a p95 of 8 ms, confirming the pre-computed ranking score tactic (ADR-004). The PATCH /api/requests/:id/accept endpoint is an outlier at 5.9s because it synchronously executes the `RankingScoreEngine.recompute()` and `updateResponseRate()` DB operations before returning. Login latency includes bcrypt password hashing (cost factor 10), which is computationally heavy by design.

Mitigation: Moving `recompute()` to a BullMQ job triggered by the domain event (instead of inline execution) would reduce the accept endpoint to < 100 ms. The frontend already applies an optimistic UI update immediately on button click, masking the server latency for the end user.

4.5.2 NFR2 – Chat Message Delivery Latency (< 100 ms end-to-end)

Chat message delivery was measured from the point a POST /api/connections/:id/messages request was sent to the point the WebSocket broadcast was received by a co-located client:

Measurement	p50 (ms)	p95 (ms)
HTTP message save to DB	9	20
WebSocket broadcast (local)	2	6
Redis pub-sub round-trip	3	8
Total end-to-end (local)	14	34

Analysis: The end-to-end local latency of 34 ms at p95 comfortably satisfies the < 100 ms requirement. Redis pub-sub adds 3–8 ms overhead in exchange for horizontal scalability across instances. On a WAN deployment (e.g., Bangalore → Mumbai), additional TCP round-trip time of 10–30 ms would still keep total latency within the target.

4.6 Trade-off Discussion

4.6.1 Synchronous Ranking Recomputation vs. Async Job

Current: Ranking recomputation is synchronous inside the accept handler. This guarantees consistency (the next directory query reflects the updated score) but causes the PATCH /accept endpoint to take 2–7 s.

Alternative: Emit a `recompute.ranking` event and process it in a BullMQ worker. Accept latency drops to < 100 ms, but ranking score lags by up to ~5 s (eventual consistency).

Decision: The frontend optimistic update already masks the accept latency from the user. The synchronous approach was retained for the prototype to guarantee correctness in the absence of a dead-letter queue for failed recomputation jobs.

4.6.2 In-Process EventBus vs. Durable Message Broker

Current: `Promise.allSettled` with in-process handlers. Failed handlers log an error and do not retry.

Alternative: Route all domain events through BullMQ (already present). Each event becomes a persistent job with retry semantics (e.g., 3 retries with exponential backoff for email dispatch).

Decision: The in-process bus provides sufficient reliability for the prototype scale. A production system would migrate email and notification dispatch to BullMQ jobs to achieve at-least-once delivery guarantees.

4.6.3 Monolith vs. Microservices (Summary)

For a university-scale deployment (expected concurrency: 50–500 simultaneous users), the layered monolith provides significantly faster development velocity and simpler operations with no material scalability penalty. The internal EventBus and Redis-backed pub-sub provide a *migration path*: each subscriber can independently be extracted into a separate service if load necessitates it, without changing the event contract.

Conclusion

AlumniConnect demonstrates a pragmatic architecture that balances modifiability, performance, and development velocity. The five key architectural decisions – Observer-based event decoupling, Strategy-pattern meeting provider, Redis pub-sub WebSocket scaling, pre-computed ranking scores, and BullMQ persistent jobs – directly trace to the system’s non-functional requirements.

Quantitative measurement confirms that read-heavy endpoints (directory, notifications, metrics) meet their p95 latency targets comfortably. The single outlier, the request acceptance endpoint, is explained by synchronous ranking recomputation and is addressable via an async job pattern already present in the infrastructure.

The layered monolith with internal event bus provides a solid foundation for university-scale deployment while preserving a clear migration path to a more distributed architecture as the platform grows.

Team Contributions

The following table documents the primary contributions of each team member across the four phases of this project.

Team Member	Roll No.	Contributions
Sriram Metla	2024201059	Task 1 (Requirements elicitation, FR/NFR documentation); subsystem decomposition; backend Auth Service and Alumni Directory implementation.
Dhanush Devireddy	2024201028	Task 2 (IEEE 42010 stakeholder analysis, ADR-001, ADR-002); Mentorship Request state machine; EventBus and RequestListeners implementation.
Chandana Pappula	2024201060	Task 2 (ADR-003, ADR-004); Task 3 (Architectural tactics); Real-Time Chat (WebSocket, Redis pub-sub, presence tracking) implementation.
Yaswanth Chandaluri	2024201045	Task 3 (Design pattern documentation); Session Scheduling Service; BullMQ job queue and reminder pipeline; Meeting Provider Strategy pattern.
AdhiVishnu Modapu	2024202028	Task 4 (Architecture analysis, NFR quantification, trade-off discussion); Admin Dashboard; Verification Pipeline (Chain of Responsibility); Health Score Service.

All team members participated in testing, code review, integration, and report writing. The above reflects primary ownership areas.
